

HW2 Classifications for diabetes (100 pts)

Upload to your Github by : 5/20/2026 (Wed: 10AM)

Midterm Exam is: 5/25/2026 (Mon, 10AM)

The diabetes_prediction_dataset.csv file contains medical and demographic data of patients along with their diabetes status, whether positive or negative.

It consists of various features such as age, gender, body mass index (BMI), hypertension, heart disease, smoking history, HbA1c level, and blood glucose level. The Dataset can be utilized to construct machine learning models that can predict the likelihood of diabetes in patients based on their medical history and demographic details.

Step 1: Load Data (5 pts)

Get Dataset:

https://www.kaggle.com/datasets/iammustafatz/diabetes-prediction-dataset?select=diabetes_prediction_dataset.csv

```
(100000, 9)
```

	gender	age	hypertension	heart_disease	smoking_history	bmi	HbA1c_level	blood_glucose_level	diabetes
0	Female	80.0	0	1	never	25.19	6.6	140	0
1	Female	54.0	0	0	No Info	27.32	6.6	80	0
2	Male	28.0	0	0	never	27.32	5.7	158	0

Notes: Diabetes is the target variable being predicted, with values of 1 indicating the presence of diabetes

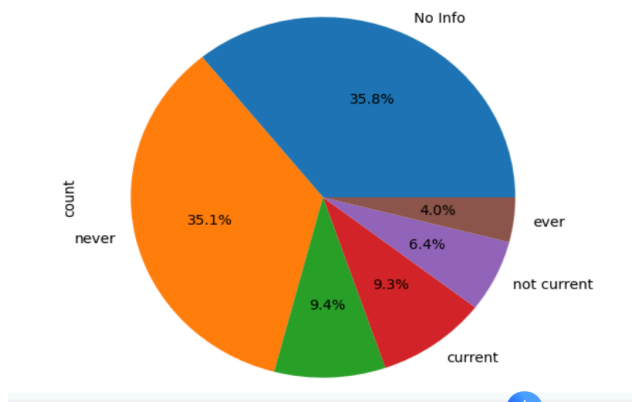
Step 2: Basic Data Analysis (10 pts)

- 1) Show basic Info
- 2) Show number rows and columns
- 3) Show column names
- 4) Show data types
- 5) Show missing data (hint: use `df.isnull().sum()`)
- 6) Show descriptive statistics of age and bmi (Use `.describe()`)

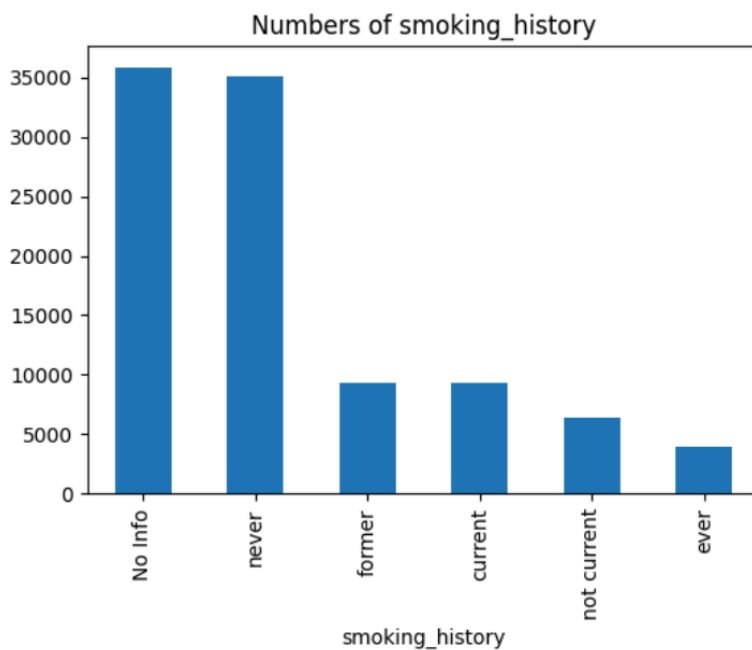
7) Show mean, std, min, max of **age** and **bmi** (Use min, max, std, mean function)

Step 3: Data Visualization: show following chart (20 pts)

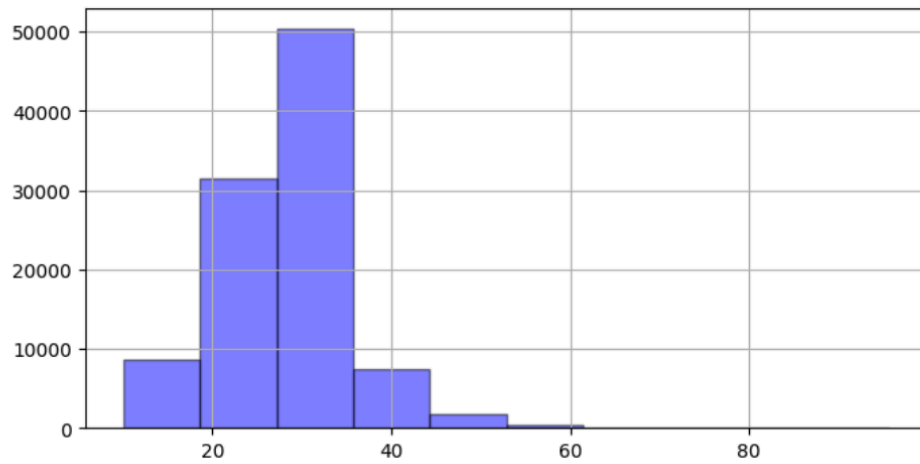
1) Create Pie chart for Smoke History



2) Bar chart for Smoke History

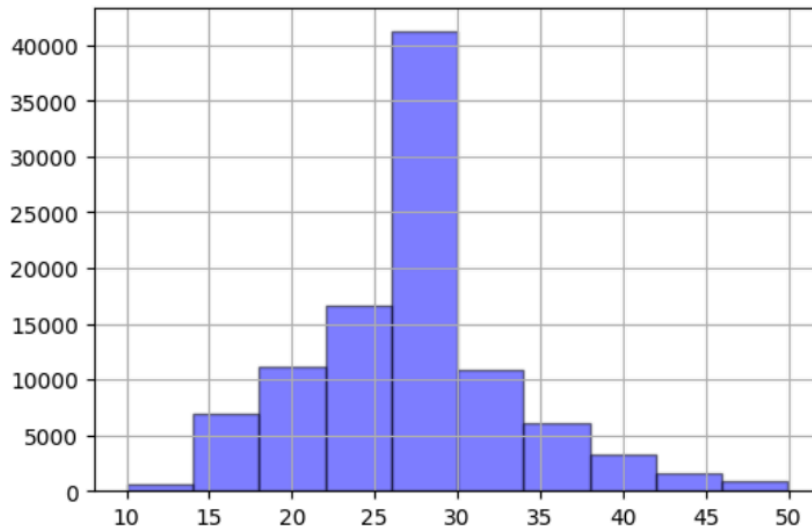


3) Histogram chart (Show all bmi)



4) Histogram chart for Smoke History (Show bmi < 60)

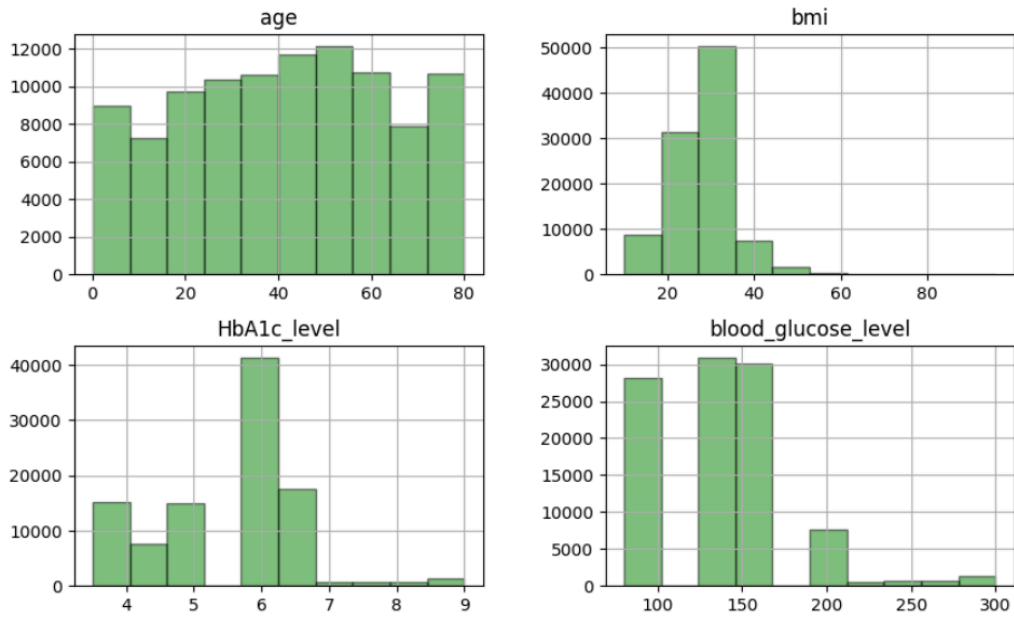
Name: bmi, dtype: float64



5) Charts by your choice

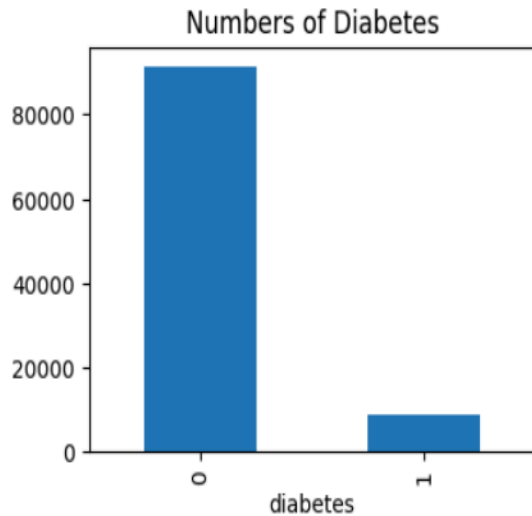
(Suggestion: charts show the correlations of age, bmi to diabetes)

...



6) Target Data

```
diabetes
0    91500
1     8500
Name: count, dtype: int64
<Axes: title={'center': 'Numbers of Diabetes'}, xlabel='diabetes'>
```



7)

Step 4: Data Processing (20 pts)

Convert categorical features to numerical feature

Method 1 : Using LabelEncoder for gender

```
print(df.gender.dtypes)
df.head(2)
```

```
... object
int64
```

	gender	age	hypertension	heart_disease	smoking_history	bmi	HbA1c_level	blood_glucose_level	diabetes
0	0	80.0	0	1	never	25.19	6.6	140	0
1	0	54.0	0	0	No Info	27.32	6.6	80	0

Method 2: Using mapping for Smoking_history

```

# Define the order
print(df.head(2))
hist_map = {'never':0, 'No Info':2, 'current':10, 'former':8, 'ever':20, 'not current':5}
# Apply mapping and get new columns
df['smoking_hist'] = df['smoking_history'].map(hist_map)
df.head(2)

```

```

...   gender  age  hypertension  heart_disease  smoking_history  bmi \
0      0  80.0             0             1         never  25.19
1      0  54.0             0             0         No Info  27.32

   HbA1c_level  blood_glucose_level  diabetes
0          6.6                 140          0
1          6.6                 80          0

```

	gender	age	hypertension	heart_disease	smoking_history	bmi	HbA1c_level	blood_glucose_level	diabetes	smoking_hist
0	0	80.0	0	1	never	25.19	6.6	140	0	0
1	0	54.0	0	0	No Info	27.32	6.6	80	0	2

Method 3: Using dummies

Df_us_dummies

```

# Only converting categoric features to numerical feature
# Method 3 | : Using get_dummies -
category_cols = ['gender', 'smoking_history']

df_use_dummies = pd.get_dummies(df, columns = category_cols , dtype=int )
print('-----')
print(df_use_dummies.dtypes)
df_use_dummies.head(2)

```

```

-----
age                                float64
hypertension                       int64
heart_disease                     int64
bmi                                float64
HbA1c_level                       float64
blood_glucose_level              int64
diabetes                          int64
gender_Female                    int64
gender_Male                      int64
gender_Other                     int64
smoking_history_No Info          int64
smoking_history_current          int64
smoking_history_ever             int64
smoking_history_former           int64
smoking_history_never            int64
smoking_history_not current      int64
dtype: object

```

Method 4: One-hot Encoder (Bonus 5 pts)

```

# # 4. Preprocessing: OneHot for categorical + MinMax scaling for numeric
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder()
encoded = ohe.fit_transform(df[['Gender']])
encoded_df = pd.DataFrame(encoded.toarray(), columns=ohe.get_feature_names_out(['gender']))

```

Step 5: Separate Data feature and target (5 pts)

```
# Current process data has not category
print(df.columns)
cols_used = ['gender', 'age', 'hypertension', 'heart_disease', 'smoking_hist',
             'HbA1c_level', 'blood_glucose_level']

# Seperate data part
data = df[cols_used]

# Get target
target = df['diabetes']

print(data.shape)
print(target.shape)
print(target.head(2) )
data.head(3)
```

```
Index(['gender', 'age', 'hypertension', 'heart_disease', 'smoking_history',
       'bmi', 'HbA1c_level', 'blood_glucose_level', 'diabetes',
       'smoking_hist'],
      dtype='object')
(100000, 7)
(100000,)
0      0
1      0
Name: diabetes, dtype: int64
```

	gender	age	hypertension	heart_disease	smoking_hist	HbA1c_level	blood_glucose_level
0	0	80.0	0	1	0	6.6	140

Step 6: Training three classifiers (40)

- Separate training and testing dataset
- Create three classifiers
- Apply testing data
- Evaluation Report
- Confusion Matrix

1) Using KNN Classifier: (Try at least three k numbers, hint: use loop)

2) Using Decision Tree

3) Third Classifier: Your Choice (I Use SVM here)

```
# Step 4: Create a SVM
from sklearn import svm
svm_classifier = svm.SVC()

#Training the decision classifier
svm_classifier.fit (X = data_train, y = target_train)
print(svm_classifier)

# Step 5: Testing
predicted = svm_classifier.predict(X= data_test)

# Step 6: Evaluation, Classification report
from sklearn.metrics import classification_report
report = classification_report(target_test, predicted)
print(report)

#Use Confusion_matrix
from sklearn.metrics import confusion_matrix
matrix = confusion_matrix(y_true = target_test, y_pred = predicted)
print('----- Confusion Matrix')
print(matrix)
```

SVC()	precision	recall	f1-score	support
0	0.95	1.00	0.97	18309
1	1.00	0.39	0.56	1691
accuracy			0.95	20000
macro avg	0.97	0.70	0.77	20000
weighted avg	0.95	0.95	0.94	20000

----- Confusion Matrix

```
[[18309    0]
 [ 1028   663]]
```

Step 7: Finishing following table: (20 pts)

(Using confusion matrix)

Diabetes is the target variable being predicted, with values of 1 indicating the presence of diabetes

Classifier	TP	FP	TN	FN	Precision	Recall	F1-score	Accuracy
<u>KNN (K=1)</u>	2089	1021	35572	1318	0.67	0.61	0.64	0.94
<u>KNN (K=3)</u>	1909	444	36149	1498	0.81	0.56	0.66	0.95
<u>KNN (K=5)</u>	1825	230	36363	1582	0.89	0.54	0.67	0.95
<u>KNN (K=7)</u>	1760	172	36421	1647	0.91	0.52	0.66	0.95
<u>Decision Tree</u>	2295	0	36593	1112	1.00	0.67	0.80	0.97
<u>Random Forest</u>	2345	116	36477	1062	0.95	0.69	0.80	0.97